Case Study 02: Vehicle Journey Fuel and Cost Estimation

1. Problem Statement

Develop a Python program to compute and compare the fuel requirements and travel expenses for different vehicles over a specified journey distance. The program must read the total trip distance and current fuel price per litre, evaluate a fleet of predefined vehicles with varying fuel efficiencies (mileage), incorporate a mandatory 10% fuel reserve overhead, compute the total journey cost for each vehicle, and determine the most economical option.

2. Objectives

- To validate user inputs for journey distance and fuel price against non-positive and negative numerical values.
- To iterate through a structured collection of vehicle specifications represented as tuples.
- To calculate fuel consumption with a 10% reserve margin and determine the monetary expenditure per vehicle.
- To identify and display the most cost-effective vehicle using Python's min() function with a lambda key selector.

3. Assumptions

- The fleet consists of three predefined vehicles with constant fuel efficiencies: Vehicle A (12 km/l), Vehicle B (15 km/l), and Vehicle C (20 km/l).

- Total fuel needed includes a 10% reserve buffer, calculated as fuel $= \left(\frac{\text{distance}}{\text{mileage}}\right) \times 1.10.$

- Journey distance must be strictly greater than 0, while fuel price must be non-negative.

- Currency calculations are rounded to two decimal places.

4. Algorithm

- **Step 1:** Start the program.

- **Step 2:** Prompt the user to enter distance (in km) and fuel_price (per litre) as floating-point numbers.

- **Step 3:** Define a list of tuples vehicles = [("Vehicle A", 12), ("Vehicle B", 15), ("Vehicle C", 20)] and initialize an empty list costs = [].

- **Step 4:** Check if distance <= 0 or fuel_price < 0. If true, print "Invalid distance or fuel price" and proceed to Step 9.

- **Step 5:** For each (name, mileage) in vehicles:

    - Compute fuel required with reserve: fuel = (distance / mileage) * 1.10.

    - Compute total cost: cost = fuel * fuel_price.

    - Append the tuple (name, cost) to costs.

    - Display the vehicle name, required fuel (rounded to 2 decimal places), and total cost (rounded to 2 decimal places).

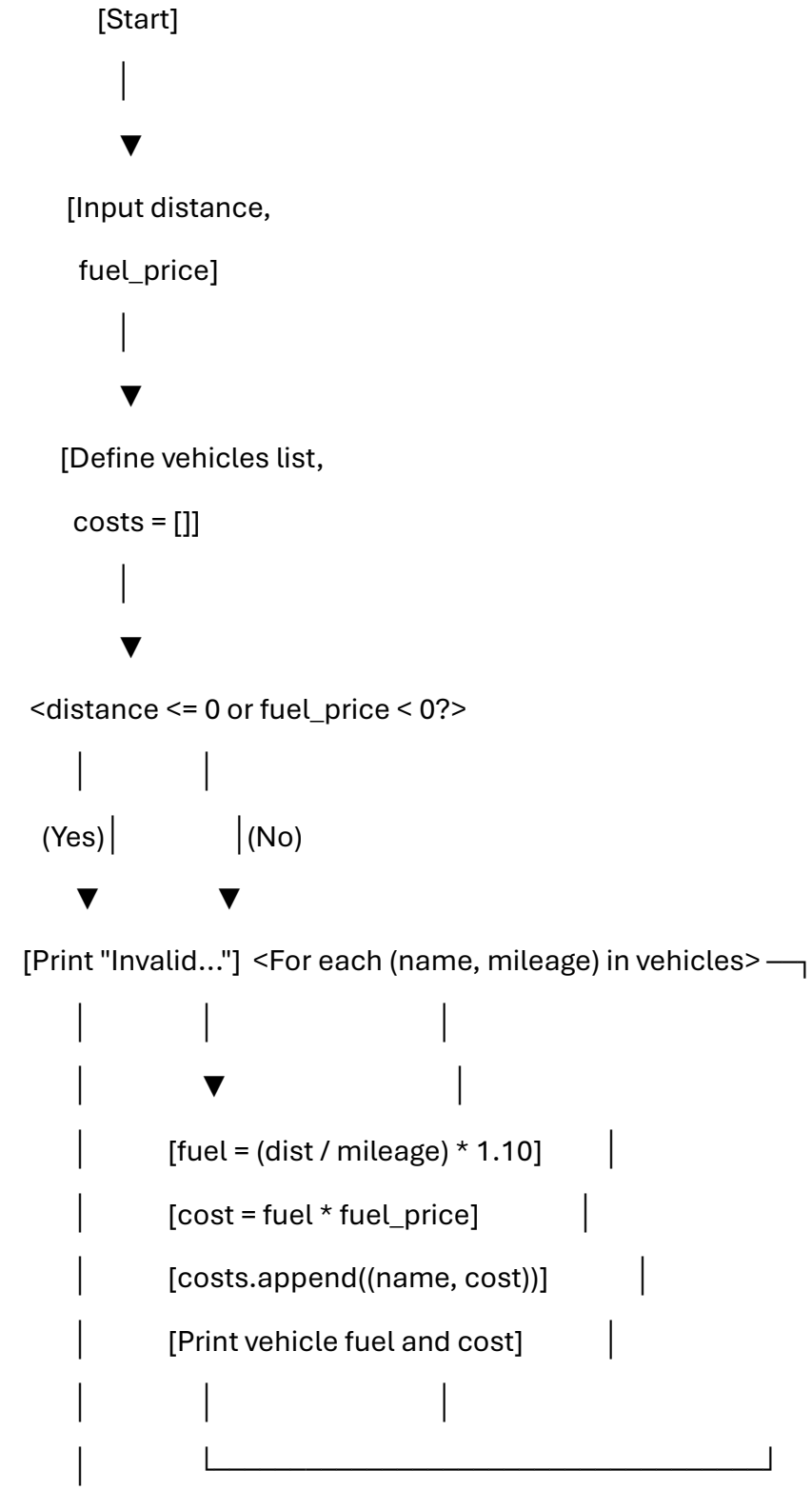
- **Step 6:** Identify the lowest-cost vehicle using min(costs, key=lambda item: item[1]) and store it in best.

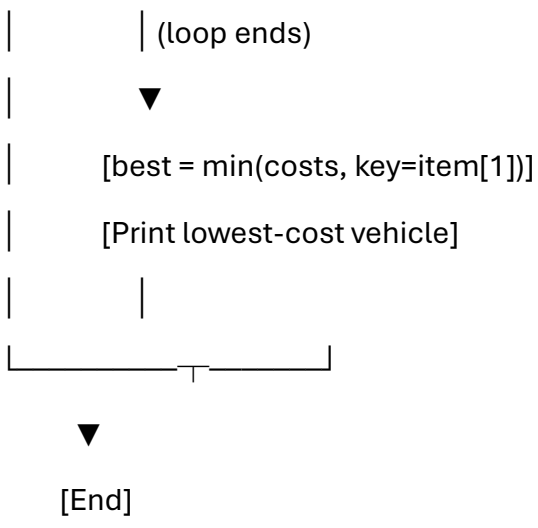
- **Step 7:** Display the name and total expense of best vehicle.

- **Step 8:** Stop the program.

5. Flowchart Logic / Diagram

Plaintext


[Start]
[Input distance,
fuel_price]
[Define vehicles list,
costs = []]
<distance <= 0 or fuel_price < 0?>
(Yes)
(No)
[Print "Invalid..."]
<For each (name, mileage) in vehicles>
[fuel = (dist / mileage) * 1.10]
[cost = fuel * fuel_price]
[costs.append((name, cost))]
[Print vehicle fuel and cost]



6. Pseudocode

```
Plaintext
BEGIN
  PRINT "Journey distance (km): "
  READ distance
  PRINT "Fuel price per litre: "
  READ fuel_price

  SET vehicles = [("Vehicle A", 12), ("Vehicle B", 15), ("Vehicle C", 20)]
  SET costs = []

  IF distance <= 0 OR fuel_price < 0 THEN
    PRINT "Invalid distance or fuel price"
  ELSE
    FOR EACH (name, mileage) IN vehicles DO
```

```
        SET fuel = (distance / mileage) * 1.10
        SET cost = fuel * fuel_price
        APPEND (name, cost) TO costs
        PRINT name, "fuel with 10% reserve:", ROUND(fuel, 2), "litres"
        PRINT name, "journey cost: Rs.", ROUND(cost, 2)
    END FOR

    SET best = MINIMUM(costs, BY cost)
    PRINT "Lowest-cost vehicle:", best[0], "Rs.", ROUND(best[1], 2)
  END IF
END
```

7. Python Source Code

```
Python
# Vehicle Journey Fuel and Cost Estimation
# Read distance and fuel rate
distance = float(input("Journey distance (km): "))
fuel_price = float(input("Fuel price per litre: "))

# Vehicle dataset with mileage (km per litre)
vehicles = [("Vehicle A", 12), ("Vehicle B", 15), ("Vehicle C", 20)]
costs = []

# Validate numerical inputs
```

```
if distance <= 0 or fuel_price < 0:
    print("Invalid distance or fuel price")
else:
    # Iterate and compute fuel with 10% reserve and costs
    for name, mileage in vehicles:
        fuel = (distance / mileage) * 1.10
        cost = fuel * fuel_price
        costs.append((name, cost))
        print(name, "fuel with 10% reserve:", round(fuel, 2), "litres")
        print(name, "journey cost: Rs.", round(cost, 2))

    # Identify the vehicle with the lowest expenditure
    best = min(costs, key=lambda item: item[1])
    print("Lowest-cost vehicle:", best[0], "Rs.", round(best[1], 2))
```

8. Test Cases

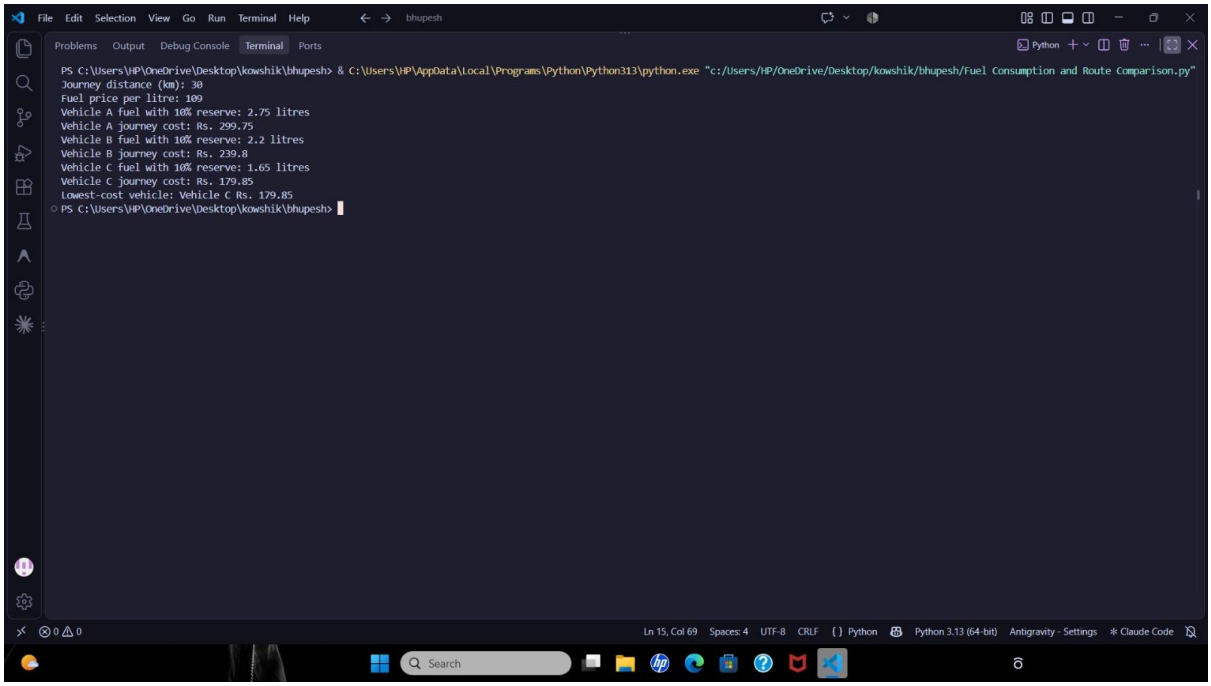
Test Case ID	Input Given	Expected Output	Actual Output	Status
TC_01 (Standard Trip)	Distance = 120 Fuel Price = 100	Vehicle A fuel: 11.0 litres, Rs. 1100.0	Vehicle A fuel with 10% reserve: 11.0 litres	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
		Vehicle B fuel: 8.8 litres, Rs. 880.0 Vehicle C fuel: 6.6 litres, Rs. 660.0 Lowest-cost: Vehicle C Rs. 660.0	Vehicle A journey cost: Rs. 1100.0 Vehicle B fuel with 10% reserve: 8.8 litres Vehicle B journey cost: Rs. 880.0 Vehicle C fuel with 10% reserve: 6.6 litres Vehicle C journey cost: Rs. 660.0 Lowest-cost vehicle: Vehicle C Rs. 660.0	
TC_02 (Long Distance)	Distance = 300	Vehicle A fuel: 27.5 litres, Rs. 2626.25	Vehicle A fuel with 10% reserve: 27.5 litres	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
	Fuel Price = 95.5	Vehicle B fuel: 22.0 litres, Rs. 2101.0 Vehicle C fuel: 16.5 litres, Rs. 1575.75 Lowest-cost: Vehicle C Rs. 1575.75	Vehicle A journey cost: Rs. 2626.25 Vehicle B fuel with 10% reserve: 22.0 litres Vehicle B journey cost: Rs. 2101.0 Vehicle C fuel with 10% reserve: 16.5 litres Vehicle C journey cost: Rs. 1575.75 Lowest-cost vehicle: Vehicle C Rs. 1575.75	

Test Case ID	Input Given	Expected Output	Actual Output	Status
TC_03 (Invalid Distance)	Distance = -50 Fuel Price = 100	Invalid distance or fuel price	Invalid distance or fuel price	Pass
TC_04 (Zero Distance)	Distance = 0 Fuel Price = 100	Invalid distance or fuel price	Invalid distance or fuel price	Pass
TC_05 (Negative Fuel Price)	Distance = 100 Fuel Price = -5	Invalid distance or fuel price	Invalid distance or fuel price	Pass

9. Output Screenshots



10. Conclusion

The Vehicle Journey Fuel and Cost Estimation program was executed and verified successfully. The script demonstrates data collection handling using lists of tuples, mathematical scaling with percentage reserves, formatted floating-point rounding, and key-based sequence minimization via lambda expressions.